`carl_simulation.py` — Full Line-by-Line Walkthrough

File: the main entry point. This is what `python carl_simulation.py` actually runs.
Length: 902 lines. This document explains every line that does something; pure blank lines and one-word closing brackets are grouped into their parent block instead of getting their own entry, since explaining `}` on its own line teaches nothing.

Lines 1–9: Module docstring

```python
"""
carl_simulation.py — EBCA Full Autonomous Simulation Runner (Modular SoC Desi

Integrates modular cognitive files using clean imports.
Unifies memory with zero-copy S_unified architecture (Apple Silicon concept).

Diagnostic ID Tracking (PHY-001..004, LSM-211..212, END-201..205,
MEM-101..104, WIT-401, MPC-511, IMA-601, CON-701, GIS-901, HAL-911)
"""
```

This is a docstring, not executable code — Python ignores it at runtime except for storing it as `__doc__`. It's telling you two design decisions up front:

1. **"SoC Design"** = System-on-Chip design. The idea: instead of one giant tangled script, each cognitive subsystem (reservoir, endocrine, reflex, grid cells) is its own file/module, imported cleanly — like separate chips on a circuit board, each with one job.
2. **"S_unified"** = one single big NumPy array holding *everything* the brain currently knows (sensors, neural state, hormone levels, motor output) instead of scattered separate variables. You'll see this built at line 429. The "Apple Silicon" comparison refers to *unified memory* — the same trick Apple's M-series chips use so the CPU and GPU don't need to copy data back and forth between separate memory pools.
3. The diagnostic IDs (`PHY-001`, `END-201`, etc.) are tags you'll see sprinkled through `print()` statements later — a lightweight way to grep your own console logs by subsystem (Physics, Endocrine, Working memory, Witness, MPC, Imagination, Consciousness, Global Information Space, Hardware Abstraction Layer) without a real logging framework.

Lines 11–24: Imports

python

```python
import sys
import os
import math
import numpy as np
import time
import traceback
import csv
import mujoco
import mujoco.viewer
import torch
import torchvision
import functools
import torch.nn as nn
from torchvision.models.detection.ssdlite import SSDLiteClassificationHead
```

- `sys`, `os` — path manipulation (used at line 26–27 to make the script portable).
- `math` — trig functions (`atan2`, `cos`, `sin`) for angle/position math throughout.
- `numpy as np` — every vector in this file (sensor arrays, motor commands, the whole "brain state") is a NumPy array, not a Python list, because NumPy lets you do whole-array math (`raw_sonar / 5.0`) in one line instead of a loop.
- `time` — wall-clock timing for the 30Hz control loop and telemetry timestamps.
- `traceback` — used in `except` blocks to print full stack traces instead of just the error message, so a crash tells you *where*, not just *what*.
- `csv` — writes the telemetry log file.
- `mujoco` / `mujoco.viewer` — the physics engine. `mujoco` runs the simulation math (rigid body dynamics, contacts, sensors); `mujoco.viewer` is the 3D visualization window.
- `torch`, `torchvision` — CARL's "eye." It runs a real object-detection neural network (SSD-Lite) on the camera image every tick.
- `functools`, `torch.nn as nn`, `SSDLiteClassificationHead` — used only inside `get_ssdlite_model()` (next section) to rebuild the final classification layer of a pretrained detector so it recognizes CARL's 4 custom classes instead of the 91 COCO classes it was originally trained on.

Lines 26–27: Portable path resolution

python

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
sys.path.append(BASE_DIR)
```

`__file__` is this script's own path. `os.path.abspath` makes it absolute (resolves `..` and symlinks), `os.path.dirname` strips the filename to get the containing folder — i.e. "whatever folder this file lives in, on whatever machine it's running on." Appending that to `sys.path` means Python will look in this folder (and its subfolders, via the next imports) when resolving `import` statements below. **This is the fix that replaced the old hardcoded `D:/ebca` path** — now it works regardless of where you clone the repo.

Lines 30–34: Brain module imports

python

```python
from brain.carl_reservoir import LiquidStateReservoir
from brain.carl_endocrine import EndocrineSystem
from brain.carl_reflex import SpikingReflexLayer
from brain.carl_grid_cells import GridCellModule, PlaceCellLayer
from torchvision.models.detection.ssdlite import SSDLite320_MobileNet_V3_Larg
```

These pull in the four cognitive subsystems this file actually wires together (there's a separate walkthrough coming for each of these — this is just "here's where they enter the picture"). `SSDLite320_MobileNet_V3_Large_Weights` is torchvision's enum of pretrained weight sets for the specific vision architecture used here.

Lines 37–50: `get_ssdlite_model()` — building the eye

python

```python
def get_ssdlite_model():
    model = torchvision.models.detection.ssdlite320_mobilenet_v3_large(
        weights=SSDLite320_MobileNet_V3_Large_Weights.DEFAULT
    )
```

Loads a pretrained SSD-Lite object detector (MobileNetV3 backbone, 320×320 input) — this is a real, standard architecture from torchvision, not custom-built. It comes pretrained on COCO (91 everyday object classes: person, car, dog, etc.) — useless for CARL as-is, since CARL only cares about food/obstacle/predator.

python

```python
    in_channels = [layer[0][0].in_channels for layer in model.head.classifica
    num_anchors = model.anchor_generator.num_anchors_per_location()
```

Digs into the model's internal structure to find how many input channels feed the classification head at each of its feature-map scales, and how many anchor boxes (candidate bounding-box shapes) are predicted per spatial location. Both numbers are needed to build a *replacement* head with the same shape but different output classes.

```python
    norm_layer = functools.partial(nn.BatchNorm2d, eps=0.001, momentum=0.03)
    model.head.classification_head = SSDLiteClassificationHead(
        in_channels=in_channels,
        num_anchors=num_anchors,
        num_classes=4,
        norm_layer=norm_layer
    )
    return model
```

Swaps out the final classification layer for a fresh one with `num_classes=4` — this is **transfer learning**: keep the pretrained backbone's learned visual features (edges, textures, shapes), but retrain only the final layer to output CARL's own categories (background + food + obstacle + predator = 4). `eps`/`momentum` are BatchNorm's standard numerical-stability and running-average smoothing constants — copied from torchvision's own reference implementation, not tuned specifically here.

Lines 57–65: `HardwareInterface` — the abstract contract

```python
class HardwareInterface:
    """Abstract interface for decoupling cognitive brain from physical/virtua
    def read_sensors(self) -> np.ndarray:
        """Returns 25-D vector mapping [Sonar (24), Battery (1)]"""
        raise NotImplementedError

    def write_motors(self, torque_left: float, torque_right: float, neck_targ
        """Applies control signals directly to physical/virtual servos"""
        raise NotImplementedError
```

This defines a **contract, not an implementation** — it says "anything calling itself a hardware interface for CARL must be able to read a 25-number sensor vector and accept 3 motor commands," but doesn't say *how*. `raise NotImplementedError` is the standard Python pattern for "if you call this on the base class directly, you did something wrong — subclass me instead." This is genuinely good design: it means when Phase 2 (real hardware) happens, you write a `RealRobotHAL(HardwareInterface)` with real GPIO/serial code, and `carl_simulation.py`'s brain logic below **never has to change** — it only ever talks to `self.hal.read_sensors()`/`self.hal.write_motors()`, not to MuJoCo directly.

`SimulationHAL` — the concrete simulation implementation (lines 68–288)

This class is the *only* thing in the file that talks to MuJoCo directly. Everything below it treats `self.hal` as a black box that returns numbers.

`__init__` (lines 70–123)

```python
    def __init__(self, xml_path: str, open_space: bool = False):
        try:
            self.model = mujoco.MjModel.from_xml_path(xml_path)
```

Loads the physics world definition — `vessel_kinetic.xml` — compiling all the bodies, joints, actuators, and sensors described in that XML into an in-memory physics model.

```python
            if open_space:
                for i in range(self.model.ngeom):
                    name = mujoco.mj_id2name(self.model, mujoco.mjtObj.mjOBJ_
                    if name and name.startswith("lwall_"):
                        self.model.geom_pos[i] = [99.0, 99.0, -99.0]
                print("[HAL-911] Open space training enabled. All internal wa
```

An optional training mode. `self.model.ngeom` is the total count of physics "geoms" (collision shapes) in the world. This loop walks every geom, checks if its name starts with `"lwall_"` (labyrinth wall), and if so, teleports it to `(99, 99, -99)` — far outside the arena and below the floor, effectively deleting it from the playable space without actually removing it from the model (removing geoms at runtime isn't simple in MuJoCo; moving them out of reach is the practical workaround). This lets you toggle between "maze mode" and "open field" for different training phases via the `--open-space` CLI flag (see line 900).

```python
self.data = mujoco.MjData(self.model)
```

`MjModel` is the static description (geometry, masses, joint limits — doesn't change). `MjData` is the *dynamic state* (current positions, velocities, forces) — this is what gets stepped forward in time every physics tick.

```python
self.cam_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOBJ_C
if self.cam_id < 0:
    raise ValueError("Error: eye_cam camera not found in XML mode
self.robot_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.
self.food_geom_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.m
self.hazard_body_id = mujoco.mj_name2id(self.model, mujoco.mjtObj
```

`mj_name2id` looks up a numeric ID from a human-readable name defined in the XML. MuJoCo's internal APIs work with integer IDs everywhere for speed, so this converts the readable names (`"eye_cam"`, `"drive_deck"`, `"food"`, `"hazard_cube"`) into the IDs used for the rest of the file. The explicit check-and-raise on `cam_id < 0` is a fail-fast guard: if the camera is missing from the XML, crash immediately with a clear message instead of getting a cryptic index error hundreds of lines later.

```python
self.cam_fov_h = 60.0
self.img_width = 160
self.img_height = 120
```

Camera parameters: 60° horizontal field of view, 160×120 pixel resolution. These are **not physics constants** — they're chosen to match a real, cheap webcam/camera module you'd actually mount on hardware in Phase 2 (low-res, wide-ish FOV), and they're reused later for the vision→sonar angle math.

```python
self.sonar_angles = np.linspace(-np.pi, np.pi, 24, endpoint=False
```

Creates 24 evenly-spaced angles from −180° to +180° (in radians), `endpoint=False` meaning it stops just short of +180° so you don't get a duplicate ray at both −180° and +180° — 24 rays spread evenly around a full circle, 15° apart.

```python
```

```
self.battery_level = 1.0
```

Starts fully charged (1.0 = 100%). This is CARL's hunger/survival pressure — see `read_sensors()` below, where it drains a tiny bit every tick.

python

```
self.head_pan_qposadr = int(self.model.joint("head_pan").qposadr[
```

Finds the memory address (`qposadr` = position address) inside MuJoCo's big flat position array where this specific joint's angle is stored, so it can be read directly later (`get_head_pan_angle()`) without a name lookup every single tick — a performance optimization since this runs 30× per second.

python

```
self.device = torch.device("cuda" if torch.cuda.is_available() el
self.vision_model = get_ssdlite_model()
weights_path = os.path.join(BASE_DIR, "memory", "carl_multi_objec
if os.path.exists(weights_path):
    self.vision_model.load_state_dict(torch.load(weights_path, ma
    print(f"[VISION] Loaded trained multi-object SSD-Lite model c
else:
    print(f"[VISION][WARNING] Trained weights not found at {weigh
self.vision_model.to(self.device)
self.vision_model.eval()
```

Picks GPU if one's available, otherwise falls back to CPU. Builds the vision model (from `get_ssdlite_model()` above), then tries to load your **own trained weights** — the result of running `brain/train_multi_object_vision.py` — on top of the pretrained backbone. If those weights don't exist yet (fresh clone, never trained), it just warns and continues with the untrained classification head — the robot will still run, just with vision that hasn't learned CARL's specific classes yet. `.eval()` puts the model in inference mode (disables dropout/batchnorm-training-behavior — standard PyTorch practice, always required before using a model to predict rather than train).

python

```
self.offscreen_renderer = mujoco.Renderer(self.model, height=120,
```

A GPU/CPU renderer that produces camera images *without* opening a visible window — needed to feed the vision model a raw pixel array every tick.

```python
            self.viewer = mujoco.viewer.launch_passive(self.model, self.data)
            self.viewer.cam.azimuth = 45
            self.viewer.cam.elevation = -32
            self.viewer.cam.distance = 7.0
            self.viewer.cam.lookat[:] = [0.0, 0.0, 0.2]
```

This *is* the visible 3D window you actually watch while it runs. `launch_passive` means the viewer doesn't own the simulation loop (you manually call `mujoco.mj_step` yourself elsewhere — see `run_simulation()` at the bottom) — it just displays whatever state `self.data` currently holds. The four camera lines are just a fixed starting viewpoint (45° rotation, tilted down 32°, 7 units back, looking near the origin) — cosmetic, no effect on the simulation itself.

```python
            self.lidar_rays = []
        except Exception as e:
            print(f"[ERROR][HAL-911] Failed to initialize SimulationHAL: {e}")
            traceback.print_exc()
            raise
```

`self.lidar_rays` starts empty — it's filled in every tick by `read_sensors()` and used only for drawing the sonar rays in the viewer (see `sync_viewer_rays()` near the bottom). The `try/except` wrapping this whole constructor means any failure during setup (bad XML, missing camera, GPU error) gets logged with a full traceback *and* still re-raised (`raise` with no argument re-throws the same exception) — so the program still crashes (correctly — you don't want to continue with a half-broken robot) but you get a readable error instead of a bare Python traceback pointing into MuJoCo's C bindings.

`_is_self_body_geom()` (lines 125–138)

```python
```

```
    def _is_self_body_geom(self, geom_id: int) -> bool:
        if geom_id < 0:
            return False
        try:
            body_id = self.model.geom_bodyid[geom_id]
            drive_deck_id = mujoco.mj_name2id(self.model, mujoco.mjtObj.mjOB
            curr = body_id
            while curr > 0:
                if curr == drive_deck_id:
                    return True
                curr = self.model.body_parentid[curr]
            return False
        except Exception:
            return False
```

This answers "did this sonar ray hit CARL's own body?" A sonar ray fired from the robot outward could technically graze the robot's own chassis geometry first if not filtered out. `geom_bodyid[geom_id]` finds which body a given collision shape belongs to. Then it walks *up the parent chain* (`body_parentid`) from that body toward the root — because a robot in MuJoCo is a tree of bodies (chassis → wheels → arms → hands, etc.), and any part of that tree ultimately traces back to `drive_deck`, the root body. If the walk ever reaches `drive_deck_id`, the geom belongs to CARL itself, so the ray should ignore that hit and keep going. The underscore prefix (`_is_self_body_geom`) is Python's convention for "private, internal helper — not part of this class's public interface."

`read_sensors()` (lines 140–183) — casting the sonar rays

python

```
    def read_sensors(self) -> np.ndarray:
        """Cast 24 sonar rays starting from the robot outer boundary, excludi
        try:
            robot_x, robot_y, robot_yaw = self.get_robot_pos()
            pnt = np.array([robot_x, robot_y, 0.13], dtype=np.float64)  # 0.1
            yaw = robot_yaw
```

Gets CARL's current 2D position and heading, builds a 3D point at the robot's (x, y) but fixed height 0.13m — the physical mounting height a real ultrasonic sensor ring would sit at on the chassis.

python

```
            geomid = np.array([-1], dtype=np.int32)
            offset = 0.16  # radius of robot to start ray outside chassis
```

`geomid` is an output buffer MuJoCo's ray-casting function will write the hit geom's ID into — has to be pre-allocated as a NumPy array because `mj_ray` writes into it by reference (a C-style out-parameter). `offset = 0.16` is CARL's approximate physical radius — rays start *outside* the chassis so they don't immediately register a hit on CARL's own hull at distance zero.

python

```
            dists = []
            self.lidar_rays = []

            for a in self.sonar_angles:
                vec = np.array([math.cos(yaw+a), math.sin(yaw+a), 0], dtype=n
                pnt_ray = pnt + vec * offset
                dist = mujoco.mj_ray(self.model, self.data, pnt_ray, vec, Nor
```

For each of the 24 preset angles: build a unit direction vector rotated by the robot's current heading plus that ray's own offset angle (so the rays sweep around relative to where CARL is actually facing, not a fixed world direction). Start the ray 0.16m out from center in that direction. `mj_ray(...)` is MuJoCo's built-in ray-casting: fire a ray from `pnt_ray` in direction `vec`, excluding the robot's own body (`self.robot_body_id`) from possible hits via the `bodyexclude` argument, and get back the distance to the first thing it hits (or a negative number if nothing was hit within range).

python

```
                if dist >= 0 and (self._is_self_body_geom(geomid[0]) or geomi
                    dist = -1.0
```

Even with the body-exclude filter, this double-checks: if the hit was still somehow part of CARL's own body, or if it hit the food object specifically (sonar shouldn't "see" food — food is meant to be sonar-transparent, see the vision-fusion comment further down), discard the hit by forcing `dist` back to −1 (meaning "nothing detected").

python

```
                if dist < 0:
                    dist_val = 5.0
                    self.lidar_rays.append((pnt, pnt + vec * 5.0, False))
                else:
                    dist_val = dist + offset
                    if dist_val > 5.0:
                        dist_val = 5.0
                        self.lidar_rays.append((pnt, pnt + vec * 5.0, False))
                    else:
                        self.lidar_rays.append((pnt, pnt + vec * dist_val, Tr

                dists.append(dist_val)
```

If nothing was hit, treat it as max range (5.0m — the sonar's effective range ceiling). If something *was* hit, add back the 0.16m starting offset to get the true distance from robot center (not from the ray's start point), then clamp to the same 5.0m ceiling. `self.lidar_rays` stores line-segment data purely for the viewer's visual overlay later (`(start_point, end_point, did_it_hit_something)` — the `True`/`False` becomes red vs. green in the 3D view).

python

```
            self.battery_level = max(0.0, self.battery_level - 0.0001)
```

Battery drains by a fixed 0.0001 per tick, floored at 0 (never negative). At 30 ticks/sec, that's a 0.003/sec drain rate — full battery to empty in roughly 333 seconds (~5.5 minutes) of continuous operation without eating. This is the actual mechanism behind CARL's "hunger" — explained further in the `EndocrineSystem` walkthrough.

python

```
            sensor_vector = np.zeros(25)
            sensor_vector[0:24] = dists
            sensor_vector[24] = self.battery_level

            return sensor_vector
        except Exception as e:
            print(f"[ERROR][PHY-003] Failed to read sensors: {e}")
            raise
```

Packs the 24 sonar distances plus the 1 battery value into the 25-element vector the `HardwareInterface` contract promised. This matches the docstring at line 60 exactly.

`read_camera()` (lines 185–207) — the actual vision inference

```python
    def read_camera(self) -> dict:
        try:
            self.offscreen_renderer.update_scene(self.data, camera="eye_cam")
            img = self.offscreen_renderer.render()
```

Renders a fresh camera-view image from CARL's eye camera, given the current physics state. This is a genuine rendered image (RGB pixel array), not a simulated abstraction.

```python
            img_tensor = torch.tensor(img, dtype=torch.float32).permute(2, 0,
```

Converts the rendered image (height × width × 3-color-channels, values 0–255) into the format PyTorch models expect: `.permute(2, 0, 1)` reorders the axes to channels × height × width, `/255.0` normalizes pixel values to the 0.0–1.0 range models are trained on, `.to(self.device)` moves it to GPU if available.

```python
            with torch.no_grad():
                predictions = self.vision_model([img_tensor])
                pred = predictions[0]
```

`torch.no_grad()` disables gradient tracking — this is inference, not training, so there's no need to remember how to backpropagate, which saves memory and compute. Runs the actual neural network forward pass on the image. `predictions[0]` because torchvision detection models expect a *batch* of images and return a list of per-image results — here the batch size is always 1.

```python
```

```
            boxes = pred["boxes"].cpu().numpy()
            labels = pred["labels"].cpu().numpy()
            scores = pred["scores"].cpu().numpy()
            valid_idx = np.where(scores >= 0.50)[0]

            return {
                "boxes": boxes[valid_idx],
                "labels": labels[valid_idx],
                "scores": scores[valid_idx]
            }
```

Pulls out bounding boxes, class labels, and confidence scores, moves them off GPU back to CPU/NumPy, and **filters to only detections the model is at least 50% confident about** (`scores >= 0.50`) — a standard confidence threshold to discard the network's noisiest, least-reliable guesses.

python

```
        except Exception as e:
            print(f"[ERROR][IMA-601] Live visual inference failed: {e}")
            return {"boxes": np.zeros((0, 4)), "labels": np.zeros(0), "scores
```

If vision fails for any reason (bad render, GPU OOM, etc.), it degrades gracefully — returns an empty detection set (shape `(0, 4)` for boxes = zero rows, 4 columns) rather than crashing the whole robot over a vision hiccup.

`write_motors()` (lines 209–224)

python

```
    def write_motors(self, speed_left: float, speed_right: float, neck_target
        try:
            v_l = float(np.clip(speed_left, -20.0, 20.0))
            v_r = float(np.clip(speed_right, -20.0, 20.0))
            head_pan_pos = float(np.clip(neck_target, -1.4, 1.4))
```

Clamps all three commanded values into safe physical ranges before sending them to the physics engine — wheel speeds capped at ±20 (presumably rad/s or an equivalent MuJoCo actuator unit), and the neck/head-pan angle capped at ±1.4 radians (~±80°) so the commanded angle can never exceed what the physical neck joint can actually reach.

python

```
            self.data.ctrl[0] = v_l
            self.data.ctrl[1] = v_r
            self.data.ctrl[4] = 0.0
            self.data.ctrl[5] = head_pan_pos

            self.data.ctrl[2:4] = 0.0
            self.data.ctrl[6:] = 0.0
        except Exception as e:
            print(f"[ERROR][PHY-001] Failed to apply motor outputs: {e}")
            raise
```

`self.data.ctrl` is MuJoCo's flat array of actuator control signals — index 0 and 1 are the left/right wheel motors, index 5 is the head-pan servo. **Indices 2–4 and 6-onward are explicitly zeroed every tick** — this is the concrete, in-code confirmation of what was flagged earlier: those are the arm/grip actuators (`act_shoulder_L/R`, `act_elbow_L/R`, `act_wrist_L/R`, `act_grip_L/R` etc. from `vessel_kinetic.xml`), and this line is *actively holding them at zero* every single tick rather than merely "forgetting" them. The arms aren't just unwired — they're explicitly commanded to stay still.

`reset_robot()` (lines 226–234)

python

```
    def reset_robot(self):
        q_addr = int(self.model.joint("spatial_identity").qposadr[0])
        self.data.qpos[q_addr : q_addr+3] = [0.0, 0.0, 0.05]
        self.data.qpos[q_addr+3 : q_addr+7] = [1.0, 0.0, 0.0, 0.0]
        self.data.qpos[q_addr+7:] = 0.0
        self.data.qvel[:] = 0.0
        self.data.ctrl[:] = 0.0
        self.battery_level = 1.0
        mujoco.mj_forward(self.model, self.data)
```

`"spatial_identity"` is a free joint (6-DOF: 3 position + 4-value quaternion rotation) that gives the robot's root body its position in the world. This resets position to `(0, 0, 0.05)` — origin, 5cm above ground — and the quaternion to `[1, 0, 0, 0]`, which is the *identity rotation* (no rotation at all — quaternion math convention: `w=1, x=y=z=0` means "facing default orientation"). Everything after that address (`q_addr+7:`) is every other joint in the robot (any arm/leg joints) — zeroed to a neutral pose. `qvel[:] = 0.0` kills all velocities (otherwise the robot would keep whatever momentum it had right into the reset). `ctrl[:] = 0.0` clears all pending motor commands. Battery gets refilled to full. `mj_forward` re-runs MuJoCo's forward kinematics/dynamics once immediately so all the

derived state (world positions, sensor readings) reflects the reset instantly rather than waiting for the next physics step.

`get_robot_pos()` (lines 236–243)

python

```python
    def get_robot_pos(self) -> tuple:
        q_addr = int(self.model.joint("spatial_identity").qposadr[0])
        x = float(self.data.qpos[q_addr])
        y = float(self.data.qpos[q_addr + 1])
        qw = self.data.qpos[q_addr + 3]
        qz = self.data.qpos[q_addr + 6]
        yaw = 2.0 * math.atan2(qz, qw)
        return x, y, yaw
```

Reads x, y position directly from the free joint's position slots. `qw` and `qz` are two components of the quaternion (positions 3 and 6 in the 7-value free-joint block: `[x, y, z, qw, qx, qy, qz]`) — since CARL only rotates around the vertical (Z) axis in this 2D-ish ground robot setup, the rotation reduces to a **quaternion-to-yaw-angle conversion**: `yaw = 2 * atan2(qz, qw)`. This formula is a simplified special case of the full quaternion-to-Euler conversion that only works because roll and pitch are assumed to be zero (a ground robot doesn't tip over) — it wouldn't be correct for a robot that could also tilt or roll.

`get_head_pan_angle()`, `get_food_pos()` (lines 245–250)

Simple direct reads: the head-pan joint's current angle from the address cached in `__init__`, and the food object's world position (a copy, so the caller can't accidentally mutate MuJoCo's internal state).

`respawn_food()` (lines 252–283) — safe random food placement

python

```python
    def respawn_food(self):
        """Spawns food in validated open space with ray-tested wall clearance
        best_pos = None
        max_clearance = -1.0

        for _ in range(50):
            cand_x = np.random.uniform(-3.8, 3.8)
            cand_y = np.random.uniform(-3.8, 3.8)
            pnt = np.array([cand_x, cand_y, 0.12], dtype=np.float64)
```

Tries up to 50 random candidate positions within a `[-3.8, 3.8]` square (slightly inside the ±4.9 boundary checked elsewhere, leaving margin) before giving up.

```python
            geomid = np.array([-1], dtype=np.int32)
            min_wall_dist = 5.0
            for angle in np.linspace(0, 2*np.pi, 8, endpoint=False):
                vec = np.array([math.cos(angle), math.sin(angle), 0], dtype=n
                dist = mujoco.mj_ray(self.model, self.data, pnt, vec, None, 1
                if dist >= 0 and dist < min_wall_dist:
                    min_wall_dist = dist
```

For each candidate spot, fires 8 rays in a full circle around it (every 45°) to check how close the nearest wall is in any direction — `min_wall_dist` tracks the closest wall hit across all 8 directions.

```python
            if min_wall_dist > 0.5:
                best_pos = [cand_x, cand_y, 0.12]
                break
            elif min_wall_dist > max_clearance:
                max_clearance = min_wall_dist
                best_pos = [cand_x, cand_y, 0.12]

        if best_pos is None:
            best_pos = [0.0, 0.0, 0.12]
```

If a candidate has more than 0.5m clearance from any wall in any direction, accept it immediately and stop searching (`break`). Otherwise, keep the *best-so-far* candidate as a fallback (`elif ... > max_clearance`) in case all 50 attempts fail to find a fully-clear spot — this guarantees food always spawns *somewhere* reasonable rather than potentially never spawning. If literally nothing worked in 50 tries (`best_pos is None`, which can only happen if the very first candidate's clearance check itself failed unexpectedly), fall back to dead center as an absolute last resort.

```python
        self.model.geom_pos[self.food_geom_id] = best_pos
        mujoco.mj_forward(self.model, self.data)
        print(f"[PHY-004] Food respawned at ({best_pos[0]:.2f}, {best_pos[1]:
```

Moves the food object to the chosen position and re-runs forward kinematics so the change takes effect immediately.

`get_hazard_pos()` (lines 285–288)

Returns the hazard cube's current world position, or a far-away dummy value `(99, 99, 99)` if no hazard body exists in this particular world (defensive coding against a variant XML that doesn't define one).

`SensorFusion` (lines 295–371) — combining vision + sonar

This is one of the more clever pieces of logic in the file, worth reading closely because it solves a real cross-modal sensing problem.

```python
class SensorFusion:
    CLASS_FOOD = 1
    CLASS_OBSTACLE = 2
    CLASS_PREDATOR = 3
```

Class constants matching the vision model's 4 output classes (class 0 is implicitly "background"/nothing, standard for object detectors).

```python
    def __init__(self, img_width: int = 160, fov_deg: float = 60.0):
        self.img_width = img_width
        self.fov_rad = math.radians(fov_deg)
```

Stores camera geometry needed to convert pixel positions into real-world angles.

```python
```

```
    def fuse(self, detections: dict, sonar_dists: np.ndarray,
             sonar_angles: np.ndarray, head_pan: float = 0.0) -> dict:
        food_vec = np.zeros(2)
        threat_vec = np.zeros(2)
        food_dist = float('inf')
        threat_dist = float('inf')
        food_angle = 0.0
        threat_angle = 0.0

        boxes = detections["boxes"]
        labels = detections["labels"]
        scores = detections["scores"]

        for i in range(len(labels)):
            lbl = int(labels[i])
            box = boxes[i]
            cx = (box[0] + box[2]) / 2.0
```

Loops through every detection the vision model produced this tick. `box` is `[x_min, y_min, x_max, y_max]` in pixel coordinates; `cx` is the horizontal center of the bounding box.

python

```
            angle_cam = (0.5 - cx / self.img_width) * self.fov_rad
            angle_body = angle_cam + head_pan
```

This is the exact formula referenced in `.agents/AGENTS.md`'s "Key Implementation Constraints." `cx / self.img_width` normalizes the box center to a 0.0–1.0 fraction across the image width. `0.5 - that` flips it so pixel-center (0.5) maps to angle 0, left edge (fraction 0) maps to +0.5, right edge (fraction 1) maps to −0.5 — i.e. positive angle = left, matching a standard right-handed rotation convention. Multiplying by `fov_rad` scales that −0.5..+0.5 range into the camera's actual angular field of view. Then **critically**, `+ head_pan` — because the camera itself is mounted on the neck, which pans independently of the body. A detection dead-center in the camera image doesn't mean "directly ahead of the robot's body" if the neck is currently turned 30° to look sideways; adding the current head-pan angle corrects for that, converting a *camera-relative* angle into a *body-relative* angle.

python

```
angle_diff = sonar_angles - angle_body
angle_diff = (angle_diff + np.pi) % (2 * np.pi) - np.pi
angle_diffs = np.abs(angle_diff)
closest_ray_idx = np.argmin(angle_diffs)
```

Finds which of the 24 sonar rays points closest to this detection's body-relative angle, so the two senses (camera and sonar) can be cross-checked against the same object. The `(x + π) % (2π) - π` pattern is a standard trick for wrapping an angle difference into the range $[-\pi, \pi]$ — without it, comparing an angle near +179° to one near −179° would incorrectly look like a huge difference when they're actually only 2° apart.

python

```
if lbl == self.CLASS_FOOD:
    box_height = max(1.0, box[3] - box[1])
    depth = 33.25 / box_height
```

For food specifically: estimate real-world distance purely from how tall the bounding box appears in the image, using the classic **pinhole camera depth formula**: `depth = (focal_length × real_object_height) / apparent_pixel_height`. The comment right above this in the source (visible in the file) states the constants: focal length ≈138.56px (derived from the 60° FOV and 160px width), real food diameter ≈0.24m, so `138.56 × 0.24 = 33.25` — the file precomputed that multiplication into the single literal `33.25` rather than showing the two source numbers inline. `max(1.0, ...)` guards against division by zero or a negative box height from a degenerate detection.

python

```
    sonar_depth = sonar_dists[closest_ray_idx]
    if sonar_depth < (depth - 0.40):
        continue
```

This is the "Cross-Modal Sonar Gate" the source comments describe: since food is physically small and sonar-transparent by design (the ultrasonic beam passes through/around it rather than bouncing back), a real food item should have the sonar ray behind it hitting whatever's *behind* the food (a wall, say) at roughly the same or greater distance than the vision-estimated depth. But if the sonar reports something noticeably *closer* than the vision estimate (more than 0.4m closer), that means there's a solid wall standing between the camera and where vision thinks the food is — meaning vision is very likely hallucinating food it's seeing through/behind a wall, an artifact of imperfect training. `continue` skips this detection entirely rather than trusting a physically impossible reading.

```python
                if depth < food_dist:
                    food_dist = depth
                    food_vec = np.array([depth * math.cos(angle_body), depth
                    food_angle = angle_body
            elif lbl in (self.CLASS_OBSTACLE, self.CLASS_PREDATOR):
                depth = min(sonar_dists[closest_ray_idx], 5.0)
                if depth < threat_dist:
                    threat_dist = depth
                    threat_vec = np.array([depth * math.cos(angle_body), dept
                    threat_angle = angle_body
```

If this food detection is closer than any food seen so far this tick, keep it (in case multiple food items are visible, always track the nearest). Converts polar (distance, angle) into a Cartesian (x, y) vector in the robot's own body frame — standard trig.
Obstacles/predators use the *sonar* distance directly (not the vision depth formula) since solid objects reflect sonar reliably, and the 0.24m food-diameter assumption wouldn't apply to them anyway.

```python
        return {
            'food_vec': food_vec, 'threat_vec': threat_vec,
            'food_dist': food_dist, 'threat_dist': threat_dist,
            'food_angle': food_angle, 'threat_angle': threat_angle,
        }
```

Returns the single nearest food and nearest threat found this tick, in both vector and distance/angle form for whichever downstream consumer needs which representation.

`TelemetryLogger` (lines 378–413)

```python
```

```
class TelemetryLogger:
    COLUMNS = [
        "tick", "wall_time_s", "x", "y", "yaw",
        "DA", "NE", "SHT", "ACh", "precision",
        "motor_L", "motor_R", "motor_neck",
        "reflex_L", "reflex_R",
        "reservoir_norm", "rls_updates",
        "food_dist", "min_sonar", "battery",
        "food_eaten_total", "surge_event",
        "cognitive_L", "cognitive_R", "cognitive_neck",
    ]
```

A fixed column schema for the CSV — this list is what `live_plotter.py` and `brain/plot_telemetry.py` both expect to parse, so changing this list without updating those readers would silently break the dashboards.

python

```
    def __init__(self, path: str, flush_interval: int = 30):
        self.path = path
        self.flush_interval = flush_interval
        self.buffer = []
        self.file = open(path, 'w', newline='')
        self.writer = csv.writer(self.file)
        self.writer.writerow(self.COLUMNS)
        self.file.flush()
        print(f"[WIT-401] Telemetry logger opened: {path}")

    def log(self, row: dict):
        self.buffer.append([row.get(c, "") for c in self.COLUMNS])
        if len(self.buffer) >= self.flush_interval:
            self.flush()

    def flush(self):
        if self.buffer:
            self.writer.writerows(self.buffer)
            self.file.flush()
            self.buffer = []

    def close(self):
        self.flush()
        self.file.close()
```

`CarlSimulationRunner` (lines 420–814) — the actual brain

This is the heart of the file — everything above exists to serve this class.

Class constants (lines 421–424)

```python
    CPG_FORWARD_BIAS = 1.5
    CPG_NECK_AMPLITUDE = 0.7      # ±40 degrees — wide enough to scan corrido
    CPG_NECK_FREQUENCY = 0.04     # slightly slower sweep for stable vision f
    FOOD_CONTACT_DIST = 0.50      # 50cm — realistic for a 22cm chassis to tr
```

"CPG" = Central Pattern Generator — a concept from neuroscience/robotics for a rhythmic signal generator that produces baseline repetitive motion (like a spinal circuit generating walking rhythm) independent of higher cognitive control. `CPG_FORWARD_BIAS = 1.5` is a constant forward drive added to both wheels every tick regardless of what the "smart" brain decides — CARL always has a baseline tendency to move forward, and cognition only steers/ modulates that baseline rather than starting from a dead stop each tick. The neck amplitude/ frequency drive a slow side-to-side head sweep (like a security camera panning) so the vision system gets to see more of the environment than just what's dead ahead. `FOOD_CONTACT_DIST = 0.50` — CARL "eats" once within 50cm of food, chosen generously larger than the robot's actual ~22cm chassis size to make contact detection forgiving rather than requiring pixel-perfect collision.

`__init__` (lines 426–480)

```python
    def __init__(self, open_space: bool = False, debug: bool = False):
        self.debug = debug
        self.S_unified = np.zeros(532)
        self.sensor_state = self.S_unified[0:25]
        self.reservoir_state = self.S_unified[25:525]
        self.neuromodulators = self.S_unified[525:529]
        self.motor_commands = self.S_unified[529:532]
```

This is the "S_unified" concept from the file's opening docstring made concrete: one single 532-element array, and the other four names (`sensor_state`, `reservoir_state`, `neuromodulators`, `motor_commands`) are **NumPy views into slices of that same array**, not copies. `25 (sensors) + 500 (reservoir neurons) + 4 (hormones) + 3 (motor outputs) = 532`. Because these are *views*, writing to `self.sensor_state[:] = ...` later actually writes directly into the corresponding slice of `self.S_unified` — no separate memory, no copying between "the sensor value" and "the sensor value as stored in unified memory." This is a genuine zero-copy design, not just a naming convention.

python

```
        self.dt = 0.033
```

Simulation timestep: 0.033 seconds ≈ 1/30th of a second — a 30Hz control loop.

python

```
        xml_path = os.path.join(BASE_DIR, "world", "vessel_kinetic.xml")
        self.hal = SimulationHAL(xml_path, open_space=open_space)
```

Instantiates the hardware abstraction layer against the primate-scout body world.

python

```
        self.reservoir = LiquidStateReservoir(
            N=500, M=37, K=3,
            spectral_radius=1.0, sparsity=0.15,
            tau=0.1, sigma_in=0.1, seed=42
        )
        reservoir_path = os.path.join(BASE_DIR, "memory", "carl_reservoir.npz
        self.reservoir.load(reservoir_path)
```

Builds the LSM reservoir with 500 internal neurons (`N`), expecting a 37-dimensional input (`M` — matches the `_build_sensory_input` method below exactly), producing 3 outputs (`K` — left wheel, right wheel, neck). `spectral_radius=1.0`, `sparsity=0.15`, `tau=0.1`, `sigma_in=0.1` are reservoir-computing hyperparameters (covered in depth in the `carl_reservoir.py` walkthrough — flagging here only that they're passed in from the caller, not hardcoded inside the reservoir class itself, so this file is where you'd tune them). `seed=42` — a fixed random seed, meaning the reservoir's internal random connectivity is **reproducible** across runs (the classic "42" is also just a common arbitrary choice, not special here). Then tries to load any previously saved trained weights from disk, degrading gracefully (per the earlier-verified `os.path.exists` guard) if none exist yet.

```python
        self.endocrine = EndocrineSystem()
        self.reflex_layer = SpikingReflexLayer(num_sensors=24, num_motor_neur
        self.sensor_fusion = SensorFusion(img_width=160, fov_deg=60.0)

        from brain.carl_grid_cells import HippocampalNavigator
        self.hippocampus = HippocampalNavigator()
        self.last_active_place_cell = -1
```

Instantiates the remaining three cognitive subsystems. Note the `HippocampalNavigator` import happens *inside* `__init__` rather than at the top of the file with the others — this is a **local import**, likely just an organizational artifact of how the file evolved rather than a deliberate design choice (there's no circular-import or lazy-loading reason visible here that would require it specifically at this point) — worth tidying up to the top-level import block for consistency, though it doesn't cause any actual bug.
`self.last_active_place_cell = -1` is a sentinel value meaning "no previous place cell yet" — used just below in `step()` to avoid recording a bogus graph edge on the very first tick.

```python
        telemetry_path = os.path.join(BASE_DIR, "memory", "telemetry_autonomc
        self.telemetry = TelemetryLogger(telemetry_path, flush_interval=30)

        self.current_tick = 0
        self.last_rls_tick = 0
        self.food_eaten_total = 0
        self.start_wall_time = time.time()

        self.neuromodulators[0] = self.endocrine.DA
        self.neuromodulators[1] = self.endocrine.NE
        self.neuromodulators[2] = self.endocrine.SHT
        self.neuromodulators[3] = self.endocrine.ACh

        self.food_memory_pos = None
        self.food_memory_timer = 0
```

Sets up bookkeeping counters, opens the telemetry file, and does the first sync of hormone levels into the `S_unified` view. `food_memory_pos`/`food_memory_timer` support a short-term "I remember where food was even though I can't see it right now" working memory — implemented in `step()`, explained below.

`_build_sensory_input()` (lines 482–496)

```python
    def _build_sensory_input(self, raw_sonar: np.ndarray, fused: dict) -> np.
        u = np.zeros(37)
        u[0:24] = 1.0 - np.clip(raw_sonar / 5.0, 0.0, 1.0)
```

Builds the 37-dimensional vector fed into the reservoir (`M=37` from the constructor). Sonar distances (0 to 5m) get inverted and normalized: `1.0 - (dist/5.0)` means *close* obstacles produce a value near 1.0 (strong signal) and *far/nothing* produces near 0.0 — a deliberate choice, since "nothing detected" (5.0m/max range) becoming a near-zero neural input is more natural for a network to learn from than a large raw number representing "nothing."

```python
        u[24:26] = np.clip(fused['food_vec'] / 5.0, -1.0, 1.0)
        u[26:28] = np.clip(fused['threat_vec'] / 5.0, -1.0, 1.0)
        u[28] = self.endocrine.DA / 3.0
        u[29] = self.endocrine.NE / 3.0
        u[30] = self.endocrine.SHT / 3.0
        u[31] = self.endocrine.ACh / 3.0
        u[32] = self.motor_commands[0] / 10.0
        u[33] = self.motor_commands[1] / 10.0
        u[34] = self.motor_commands[2] / 0.7
        u[35] = 0.0
        u[36] = self.sensor_state[24]
        return u
```

Indices 24–25: food position vector, normalized by the same 5m range. 26–27: threat vector, same. 28–31: current hormone levels, scaled down by 3.0 (their typical operating range, per the endocrine model) so they sit roughly in the same numeric scale as everything else feeding a neural network — an important practical detail, since neural nets are sensitive to input scale, and mixing raw hormone values (which might range 0–3) with raw sonar (0–5) with raw motor commands (up to ±20) unscaled would make some inputs dominate others just by coincidence of unit choice, not because they're more important. 32–33: the *previous* tick's own motor commands are fed back in as input — meaning the reservoir gets to "feel" its own recent output, which is a standard recurrent/proprioceptive-feedback trick. 34: previous neck command, scaled by its own amplitude range (0.7). **Index 35 is hardcoded to `0.0`** — this is a reserved/unused input slot, presumably left as a placeholder for a future signal that hasn't been wired in yet. Index 36: current battery level, unscaled (already 0–1).

`_compute_rls_target()` (lines 498–502)

```python
    def _compute_rls_target(self, y_cognitive: np.ndarray,
                            y_executed: np.ndarray) -> np.ndarray:
        reflex_dev = y_executed - y_cognitive
        reflex_dev[2] = 0.0
        return reflex_dev
```

"RLS" = Recursive Least Squares, the online learning algorithm used to train the reservoir's output weights (detailed in the reservoir walkthrough). This method computes what the reservoir *should have* output, for training purposes: the difference between what actually got executed (cognitive output plus reflex overrides plus navigation) and what the pure cognitive layer alone produced. The idea: if reflexes had to kick in and override the cognitive layer's decision, that deviation is itself useful training signal — teaching the reservoir to eventually produce outputs closer to what the reflex system had to correct toward, so cognition gradually "absorbs" what reflexes currently have to patch. Index 2 (neck) is explicitly zeroed out of this — the neck's cognitive output isn't corrected by this signal, presumably because the neck's CPG-driven sweep isn't something reflexes ever override in the first place.

`step()` (lines 504–814) — one tick of the entire control loop

This is the single most important method in the whole codebase — it runs 30 times a second and is where every subsystem actually gets exercised together.

A. Sensing (lines 506–533)

```python
            sensor_readings = self.hal.read_sensors()
            self.sensor_state[:] = sensor_readings
            raw_rays = sensor_readings[0:24]
```

Pulls the 25-element sensor vector, writes it into the `S_unified` view (so the whole unified-memory array reflects the newest sensor data), and pulls out just the sonar portion.

```python
```

```
        sonars = np.zeros(12)
        for i in range(12):
            center_idx = 2 * i
            left_idx = (center_idx - 1) % 24
            right_idx = (center_idx + 1) % 24
            sonars[i] = min(raw_rays[left_idx], raw_rays[center_idx], raw

        raw_sonar = np.zeros(24)
        for i in range(12):
            raw_sonar[2 * i] = sonars[i]
            next_val = sonars[(i + 1) % 12]
            raw_sonar[2 * i + 1] = (sonars[i] + next_val) / 2.0
```

This is the "12 real ultrasonic sensors interpolated to 24 virtual rays" mentioned in `AGENTS.md`'s implementation constraints. First it *downsamples* the 24 fine-grained simulation rays into 12 coarser sonar "cones" by taking the minimum (closest/most conservative reading) across each cone's 3 nearest rays — modeling a real 12-sensor physical array where each sensor has a 30° cone of detection rather than a single pencil-thin beam. Then it *upsamples back to 24* by putting each of the 12 real readings at even indices and linearly interpolating between neighbors for the odd indices — this simulates what a real 12-sonar hardware array's actual angular resolution would look like once you factor in the 30°-cone geometry, rather than pretending the simulation's 24 free, perfectly precise rays are what real hardware would give you. It's a deliberate **fidelity-matching step**: keep the sim's actual sensing model honest to what Phase 2 hardware could really deliver.

python

```
        detections = self.hal.read_camera()
        head_pan = self.hal.get_head_pan_angle()
        fused = self.sensor_fusion.fuse(
            detections, raw_sonar, self.hal.sonar_angles, head_pan=head_p
        )
        if self.debug and len(detections["labels"]) > 0:
            print(f"  [DEBUG-SENSORS] head_pan: {head_pan:.3f} | fused: f

        food_is_visible = fused['food_dist'] < 5.0 and np.linalg.norm(fus
```

Runs the actual vision inference, gets the current neck angle, and fuses vision with the *degraded, hardware-realistic* sonar (not the raw 24-ray version) — meaning the cross-modal sonar gate in `SensorFusion.fuse()` is checking against the same coarse sensing a real robot would actually have, which is the right choice for training something meant to

transfer to hardware. `food_is_visible` requires both a finite distance under the 5m sonar ceiling *and* a non-trivial vector magnitude (>0.05) — the second check guards against a degenerate all-zero food_vec (e.g. if `food_dist` is somehow the default value but no real detection happened) being misread as "food detected at distance zero."

Spatial working memory (lines 537–560):

```python
            camera_saw_food = fused['food_dist'] < 5.0 and np.linalg.norm(fus
            if camera_saw_food:
                robot_x, robot_y, robot_yaw = self.hal.get_robot_pos()
                rel_x, rel_y = fused['food_vec']
                glob_x = robot_x + rel_x * math.cos(robot_yaw) - rel_y * math
                glob_y = robot_y + rel_x * math.sin(robot_yaw) + rel_y * math
                self.food_memory_pos = np.array([glob_x, glob_y])
                self.food_memory_timer = 90  # 3 seconds memory lock
```

When food is genuinely visible this tick, its body-relative position gets rotated into *world*-frame coordinates using a standard 2D rotation matrix (this is the textbook formula for rotating a vector by angle `robot_yaw`) and stored. `food_memory_timer = 90` — at 30 ticks/sec, 90 ticks = 3 seconds. This is short-term spatial memory: "I saw food over there a moment ago, even though the neck has since panned away and I can't currently see it."

```python
            elif self.food_memory_timer > 0 and self.food_memory_pos is not N
                self.food_memory_timer -= 1
                robot_x, robot_y, robot_yaw = self.hal.get_robot_pos()
                glob_x, glob_y = self.food_memory_pos
                dx = glob_x - robot_x
                dy = glob_y - robot_y
                rel_x = dx * math.cos(robot_yaw) + dy * math.sin(robot_yaw)
                rel_y = -dx * math.sin(robot_yaw) + dy * math.cos(robot_yaw)
                fused['food_vec'] = np.array([rel_x, rel_y])
                fused['food_dist'] = math.sqrt(dx**2 + dy**2)
                fused['food_angle'] = math.atan2(rel_y, rel_x)
                if self.food_memory_timer == 0:
                    self.food_memory_pos = None
```

If food *isn't* currently visible but there's still time left on the memory timer, do the *inverse* rotation: convert the remembered world-frame position back into the robot's current body-relative frame (which changes every tick as the robot moves/turns), and *overwrite*

`fused['food_vec']`/`food_dist`/`food_angle` with this remembered estimate — meaning downstream code (the `EXPLOIT` behavior mode below) can't tell the difference between "genuinely seeing food right now" and "confidently remembering where it just was," which is the intended effect: CARL keeps heading toward food it briefly lost sight of instead of immediately giving up. Once the timer hits zero, the memory is cleared.

```python
food_is_visible = fused['food_dist'] < 5.0 and np.linalg.norm(fus
```

Recomputed after the memory logic runs, since `fused` may have just been overwritten by the remembered position above.

```python
if fused['food_dist'] < 1.0 and np.linalg.norm(fused['food_vec'])
    food_ray_idx = int(round((fused['food_angle'] - (-np.pi)) / (
    for offset in range(-1, 2):
        raw_sonar[(food_ray_idx + offset) % 24] = 5.0
```

Sensory gating / selective attention: when food is very close (<1m), find the sonar ray index closest to the food's angle, and forcibly clear that ray (and its immediate neighbors, `offset` from −1 to +1) back to max range (5.0m = "nothing detected"). This exists because without it, the sonar array would report the food object itself as a nearby "obstacle" the moment CARL gets close enough to eat it, which could trigger obstacle-avoidance reflexes that push CARL *away* from the food it's trying to reach — actively counterproductive. This line is a targeted fix for exactly that conflict.

C. Safeguards (lines 568–581)

```python
robot_x, robot_y, robot_yaw = self.hal.get_robot_pos()
food_pos = self.hal.get_food_pos()
food_dist_gt = math.sqrt((robot_x - food_pos[0])**2 + (robot_y -

out_of_bounds = (abs(robot_x) > 4.9 or abs(robot_y) > 4.9)
if out_of_bounds:
    print(f"[WARNING][HAL-911] Boundary breach. Teleporting to sa
    self.hal.reset_robot()
    self.endocrine.surge_ne(0.5)
    return
```

`food_dist_gt` = "ground truth" distance to food — computed directly from actual physics positions, not from vision/sonar estimates. This is used for the actual eating-trigger logic later (line 609) rather than the possibly-noisy or possibly-memory-based `fused['food_dist']` — a deliberate choice to make "did I actually reach the food" authoritative and sensor-noise-proof, while still letting the *cognitive/behavioral* decisions upstream run on the (intentionally imperfect) sensed/remembered estimate. The boundary check (`±4.9`, just inside the `±3.8` food-spawn zone and presumably close to the arena's actual wall) catches CARL somehow ending up outside the intended play area (a physics glitch, an exploit in reflex logic, etc.) and forcibly resets rather than letting the robot get permanently stuck or fall through geometry — `return` here **skips the rest of this tick entirely**, so no motor commands get sent and no telemetry gets logged for a tick that just triggered an emergency reset.

```python
            if np.isnan(self.S_unified).any() or np.isinf(self.S_unified).any
                self.S_unified[:] = np.nan_to_num(self.S_unified, nan=0.0, po
```

A blanket numerical-stability safety net: if *anything* in the entire unified memory array has gone `NaN` or infinite (which can happen from a division-by-zero edge case, an unstable reservoir dynamic, etc.), replace all bad values in one shot rather than letting corrupted numbers propagate through every downstream calculation and silently break the whole system.

D. Hippocampal update (lines 583–598)

```python
            v_fwd = (self.motor_commands[0] + self.motor_commands[1]) / 2.0 *
            vx = v_fwd * math.cos(robot_yaw)
            vy = v_fwd * math.sin(robot_yaw)

            hip_data = self.hippocampus.step(robot_x, robot_y, vx, vy, self.c
            active_place_idx, _ = self.hippocampus.places.peak_cell()
```

Estimates forward velocity from the average of last tick's two wheel commands (a crude proxy, not measured physics velocity), scaled by an arbitrary `0.05` conversion factor, then decomposes it into world-frame x/y velocity components using the robot's heading — this feeds the grid-cell path-integration system (covered in the `carl_grid_cells.py` walkthrough), which needs a velocity estimate to update its internal "dead reckoning" sense of movement. `peak_cell()` finds which single place cell currently has the strongest firing rate — i.e., "which location does the hippocampus currently believe it's at."

```python
            if self.last_active_place_cell >= 0:
                self.hippocampus.places.update_graph(
                    self.last_active_place_cell, active_place_idx, self.curre
                )
            self.last_active_place_cell = active_place_idx

            self.hippocampus.places.diffuse_rewards(gamma=0.9, decay=0.99995,
```

Records a graph edge between the previous and current active place cell (building a topological map of "which places connect to which," skipped on the very first tick since there's no previous cell yet — this is what `last_active_place_cell = -1` in `__init__` was for). Then runs reward diffusion — spreading reward signal through the graph so places near a rewarded location gradually accumulate some "value" too (`gamma=0.9` is a discount factor, `decay=0.99995` slowly fades old reward signal over time, `n_sweeps=5` runs the diffusion process 5 times per tick for faster convergence).

Neuromodulator triggers (lines 600–629)

```python
            min_sonar = np.min(raw_sonar)
            if min_sonar < 0.2:
                self.endocrine.surge_ne(0.5)
            elif min_sonar < 0.4:
                self.endocrine.surge_ne(0.2)
```

If the closest obstacle in any direction is under 0.2m, trigger a strong norepinephrine surge (alarm/alertness response); a more distant-but-still-close obstacle (0.2–0.4m) triggers a milder surge. Two-tier threat response, not just binary.

```python
```

```
            ate_food_this_tick = False
            if food_dist_gt < self.FOOD_CONTACT_DIST:
                ate_food_this_tick = True
                self.food_eaten_total += 1
                self.food_memory_pos = None
                self.food_memory_timer = 0
                self.endocrine.surge_da(0.7)
                self.endocrine.surge_sht(0.3)
                self.hippocampus.places.stamp_reward(active_place_idx, amount
                print(f"[END-201] CARL ate food! Total eaten: {self.food_eate
                self.hal.respawn_food()
                self.hal.battery_level = 1.0
```

This is the eating event, gated on the *ground-truth* distance (not vision/memory). On contact: increment the counter, clear the food memory (no longer needed — it's been eaten), surge dopamine (reward) and serotonin (satiation/calm) hormones, stamp a strong reward value (5.0) into the hippocampal graph at the current location (so this spot gets remembered as valuable for future replay-navigation), respawn a new food item elsewhere, and fully refill the battery. Note `ate_food_this_tick` is set but never actually read anywhere else in the method — this looks like a **leftover/unused variable**, likely from an earlier version of the code that used it for something (e.g., conditionally skipping some other block) that's since been removed or refactored away without cleaning this line up. Harmless, but dead code.

python

```
            hazard_pos = self.hal.get_hazard_pos()
            hazard_dist = math.sqrt((robot_x - hazard_pos[0])**2 + (robot_y -
            if hazard_dist < 0.5:
                self.endocrine.surge_ne(0.3)

            if self.sensor_state[24] < 0.3:
                self.endocrine.surge_ne(0.05)


            self.endocrine.step(dt=self.dt)
```

Proximity to the hazard cube also triggers alertness. Low battery (`sensor_state[24]` is the battery slot in the unified sensor view, under 0.3 = under 30%) triggers a small continuous alertness nudge — modeling low-energy stress. Then `self.endocrine.step(dt)` actually advances the hormone simulation forward one timestep (decay toward baseline, etc. — covered in that file's own walkthrough) — all the

`surge_*` calls above just *add* to current levels; this line is what actually evolves them over time.

```python
self.neuromodulators[0] = self.endocrine.DA
self.neuromodulators[1] = self.endocrine.NE
self.neuromodulators[2] = self.endocrine.SHT
self.neuromodulators[3] = self.endocrine.ACh
```

Re-syncs the `S_unified` view with the freshly-stepped hormone values.

E. Reservoir step (lines 639–646)

```python
pi_sensory = self.endocrine.get_precision_weight()
u_raw = self._build_sensory_input(raw_sonar, fused)
u_weighted = pi_sensory * u_raw

noise_scale = self.endocrine.get_exploration_noise_scale()
y_cognitive = self.reservoir.step(u_weighted, noise_scale, self.c
self.reservoir_state[:] = self.reservoir.x
```

`pi_sensory` is a Bayesian-inspired "precision weight" (how much to trust incoming sensory data right now, based on current hormone state — e.g. acetylcholine typically *increases* attentional gain to sensory input in real neuroscience, which is presumably reflected in how `get_precision_weight()` is computed). The whole 37-dim sensory input gets scaled by this single scalar before reaching the reservoir — meaning the network's *effective* sensitivity to the world changes dynamically with internal hormonal state, not just the raw numbers. `noise_scale` — exploration noise, likely scaled by norepinephrine (stress/arousal tends to increase exploratory/erratic behavior) — gets passed into the reservoir step so the network's output has some randomness proportional to current arousal. `y_cognitive` is the reservoir's 3-value output (left wheel, right wheel, neck) — the "smart" layer's raw proposal for what to do, before any reflex overrides or CPG baseline gets added. `self.reservoir.x` is the reservoir's internal 500-neuron state, copied into the unified memory view.

F. Behavioral governor + motor mixing (lines 648–734)

```python
```

```
            mode = self.endocrine.get_behavioral_mode(
                battery_level=self.hal.battery_level,
                food_visible=food_is_visible
            )
```

Determines CARL's current high-level behavioral mode (`EXPLOIT`/`SURVIVE`/`EXPLORE`) purely from battery level and whether food is currently visible — this is the "endocrine behavioral governor" mentioned in the section header, a rule-based mode switch sitting on top of the continuous reservoir/reflex outputs.

python

```
            reservoir_bias = np.zeros(2)
            reflex_outputs = self.reflex_layer.step(
                sensor_dists=raw_sonar,
                reservoir_bias=reservoir_bias,
                da=self.endocrine.DA,
                food_dist=fused['food_dist']
            )
```

Runs the spiking reflex layer (fast, hard-wired obstacle-avoidance reactions — detailed in its own walkthrough). Note `reservoir_bias = np.zeros(2)` is passed in as an always-zero placeholder — meaning the reflex layer's design *supports* being biased by the cognitive reservoir's output, but that channel is currently unused/disconnected here. Another small "built but not fully wired" detail, consistent with the project's own honest self-assessment in `AGENTS.md`.

python

```
            nav_taxis = np.zeros(2)
            cpg_neck_amp = self.CPG_NECK_AMPLITUDE

            replay_nav = np.zeros(2)
            stats = self.hippocampus.places.get_graph_stats()
            max_nav = stats['max_nav_value']
            replay_blend = self.endocrine.get_replay_blend_weight(
                self.hal.battery_level, max_nav
            )
```

Sets up defaults, then computes `replay_blend` — a weight (0.0–1.0, presumably) controlling how much the hippocampal replay-navigation system should influence movement right now, based on battery level (hungrier = more reliance on remembered good locations) and how strong the best-known nav signal currently is.

```python
            if replay_blend > 0.01:
                replay_target = self.hippocampus.places.get_replay_target(act
                rtx, rty = None, None
                if replay_target is not None:
                    rtx, rty, r_val = replay_target
                else:
                    val_sum = np.sum(self.hippocampus.places.nav_values)
                    if val_sum > 0.001:
                        target_center = np.average(
                            self.hippocampus.places.centres, axis=0,
                            weights=self.hippocampus.places.nav_values
                        )
                        rtx, rty = target_center[0], target_center[1]
```

If replay navigation matters at all this tick: try to get a specific "replay target" from the current place cell's neighbors in the graph. If there's no direct neighbor target available, fall back to a *weighted average* of every place cell's center position, weighted by each cell's current nav-value — essentially "if I don't know the specific next step, at least head in the general direction of where reward tends to be, weighted by how strongly each remembered spot matters."

```python
                if rtx is not None:
                    dx = rtx - robot_x
                    dy = rty - robot_y
                    target_angle = math.atan2(dy, dx) - robot_yaw
                    target_angle = (target_angle + math.pi) % (2 * math.pi) -
                    turn_drive = float(np.clip(target_angle * 3.5, -3.5, 3.5)
                    fwd_boost = float(max(0.2, 1.2 * (1.0 - abs(target_angle)
                    replay_nav[0] = (fwd_boost - turn_drive) * replay_blend
                    replay_nav[1] = (fwd_boost + turn_drive) * replay_blend
```

Standard "steer toward a target point" logic: compute the angle from current heading to the target, wrap it into $[-\pi, \pi]$, scale it into a turn-drive signal (positive angle = target is to the left → turn left by increasing the differential between wheels), and a forward-boost term that's largest when facing the target directly (`abs(target_angle)` near 0) and shrinks (but never below a floor of 0.2) as the target gets more to the side or behind. This is a classic **differential-drive steering** pattern: `[fwd - turn, fwd + turn]` — turning left means slowing the left wheel relative to the right, or equivalently boosting the right wheel relative to the left (which is exactly what this formula does).

```python
            if mode == "EXPLOIT":
                hunger = 1.0 + 2.0 * (1.0 - self.hal.battery_level)
                satiation = 1.0 / max(0.5, self.endocrine.DA)
                motivation = hunger * satiation

                dist_scale = float(np.clip(fused['food_dist'] / 2.0, 0.3, 1.0
                turn_drive = float(np.clip(fused['food_angle'] * 4.0 * dist_s
                fwd_boost = float(max(0.0, 1.5 * min(motivation, 2.0) * (1.0
                nav_taxis[0] = fwd_boost - turn_drive
                nav_taxis[1] = fwd_boost + turn_drive
```

The direct-approach-to-visible-food behavior. `hunger` scales from 1.0 (full battery) up to 3.0 (empty battery) — the hungrier CARL is, the more motivated. `satiation` is an *inverse* dopamine term: dopamine here represents recent reward/satisfaction, so higher DA (recently fed, feeling good) *dampens* motivation (`1/DA`, floored so DA never divides by less than 0.5 to avoid an extreme spike). `motivation = hunger × satiation` combines both. The turn/ forward drive follow the same differential-steering pattern as replay-nav above, but using the food's actual angle/distance and modulated by `motivation` and `dist_scale` (less aggressive steering correction the farther away food is, since a small angle error matters less at range than up close). Notably, in this mode, `replay_nav` is computed above but **never added into** `nav_taxis` — the comment even says so explicitly ("replay nav is suppressed"): when food is directly visible/remembered, CARL just goes straight for it and ignores the more diffuse "generally head toward good areas" signal entirely.

```python
            elif mode == "SURVIVE":
                cpg_neck_amp = 0.9
                nav_taxis = replay_nav
```

Critical-hunger mode: widen the neck's scanning sweep (more visual search area) and let replay-navigation fully take over movement — go toward wherever the hippocampal map says food has historically been found, since there's no visible food right now and survival is urgent.

```python
```

```
        elif mode == "EXPLORE":
            cpg_neck_amp = 0.8
            novelty_pos = self.hippocampus.places.get_novelty_target(acti
            novelty_taxis = np.zeros(2)
            if novelty_pos is not None:
                dx = novelty_pos[0] - robot_x
                dy = novelty_pos[1] - robot_y
                target_angle = math.atan2(dy, dx) - robot_yaw
                target_angle = (target_angle + math.pi) % (2 * math.pi) -
                turn_drive = float(np.clip(target_angle * 1.8, -2.0, 2.0)
                novelty_taxis[0] = 0.5 - turn_drive
                novelty_taxis[1] = 0.5 + turn_drive

            nav_taxis = (1.0 - replay_blend) * novelty_taxis + replay_nav
```

Normal/comfortable-battery, no-food-visible mode: get a target that leads toward under-explored territory (the hippocampus's novelty-tracking logic), steer toward it with a gentler turn gain (1.8 vs 3.5/4.0 in the other modes — exploration is meant to be more relaxed/less urgent), then blend it with replay-nav: `(1 - replay_blend) × novelty + replay_nav` — as replay_blend rises (getting hungrier even in explore mode), the blend smoothly shifts weight away from pure novelty-seeking and toward the replay signal, giving a continuous rather than abrupt transition between exploration and hunger-driven navigation.

python

```
        total_reflex = reflex_outputs + nav_taxis
```

Combines the fast reflex layer's output with whichever navigation signal the current mode produced — additive combination, meaning reflexes (e.g., "turn away, wall very close") stack on top of, rather than replace, the higher-level navigational intent.

H. Motor actuation (lines 736–755)

python

```
        cpg_left = self.CPG_FORWARD_BIAS
        cpg_right = self.CPG_FORWARD_BIAS
        cpg_neck = cpg_neck_amp * math.sin(self.current_tick * self.CPG_N

        u_left = cpg_left + y_cognitive[0] + total_reflex[0]
        u_right = cpg_right + y_cognitive[1] + total_reflex[1]
        u_neck = cpg_neck + y_cognitive[2]
```

The final motor command is **layered**: a constant forward baseline (CPG) + the reservoir's learned cognitive contribution + the combined reflex/navigation signal. The neck similarly gets a sinusoidal sweep (this is where `cpg_neck_amp`, set differently per behavioral mode above, actually takes effect: `sin(tick × frequency)` produces a smooth oscillation, and its amplitude widens or narrows depending on mode) plus the reservoir's own neck contribution.

```python
            y_executed = np.array([u_left - cpg_left, u_right - cpg_right, u_

            self.motor_commands[0] = u_left
            self.motor_commands[1] = u_right
            self.motor_commands[2] = u_neck

            self.hal.write_motors(
                speed_left=u_left,
                speed_right=u_right,
                neck_target=u_neck
            )
```

`y_executed` strips the fixed CPG baseline back out — this is "what got added on top of the baseline," used later as the RLS training target (see `_compute_rls_target` above, which takes this exact value). The final commands get stored into the unified memory view and sent to the hardware layer, which will clip them into safe ranges (as seen earlier in `write_motors`).

I. Online learning schedule (lines 757–768)

```python
            has_reflex = np.linalg.norm(total_reflex) > 0.05
            has_surprise = (self.endocrine.NE > 0.15 or self.endocrine.DA > 1

            if has_reflex or has_surprise:
                time_since_rls = self.current_tick - self.last_rls_tick
                error_norm = np.linalg.norm(self.reservoir.last_prediction_er

                if time_since_rls >= 6 or (error_norm > 0.25 and time_since_
                    y_target = self._compute_rls_target(y_cognitive, y_execut
                    self.reservoir.rls_update(y_target)
                    self.last_rls_tick = self.current_tick
```

The reservoir's output weights **don't get retrained every single tick** — that would be computationally wasteful and could make learning unstable (chasing every tiny fluctuation). Instead, training only happens when there's a *reason to learn*: either something reflex- worthy happened (an actual behavioral correction was needed, `> 0.05` threshold) or a "surprise" hormonal spike occurred (elevated norepinephrine/dopamine/acetylcholine — these specific thresholds, `0.15`/`1.05`/`0.7`, are presumably tuned against each hormone's typical range). Even then, it further rate-limits: only update if at least 6 ticks have passed since the last update, *or* if the reservoir's own recent prediction error is notably high (`>0.25`) and at least 3 ticks have passed (a shorter cooldown when things are going obviously wrong, letting it correct faster). This whole block is a genuinely thoughtful piece of engineering — gating expensive/potentially destabilizing learning updates behind meaningful triggers rather than running them unconditionally.

J & K. Diagnostics + telemetry (lines 770–808)

```python
            if self.current_tick % 30 == 0:
                diag_stats = self.hippocampus.places.get_graph_stats()
                print(f"[TICK {self.current_tick:5d}] MODE:{mode:<7s} pos:({
                      f"| {self.endocrine} "
                      f"| food:{food_dist_gt:.2f}m eaten:{self.food_eaten_tot
                      f"| edges:{diag_stats['edges']} nav:{diag_stats['max_na
                      f"| cog:[{y_cognitive[0]:.2f},{y_cognitive[1]:.2f}] "
                      f"| sonar_min:{min_sonar:.2f}")
```

A console status line printed once per second (every 30 ticks at 30Hz), summarizing the current mode, position, hormone state (via `EndocrineSystem.__repr__`, likely), food distance/count, hippocampal graph stats, cognitive output, and closest sonar reading — a compact human-readable heartbeat for watching the sim run without drowning in per-tick spam.

```python
            self.telemetry.log({...})
            self.endocrine.last_surge_event = ""
            self.current_tick += 1
```

Logs the full detailed row for this tick to CSV (every tick, unlike the console summary which throttles to 1/sec) — this is the data `live_plotter.py` and `plot_telemetry.py` read. `self.endocrine.last_surge_event = ""` resets a one-shot event label after logging it, so the *next* tick's log row doesn't repeat "food eaten" or similar events that only actually happened once. Increments the tick counter for the next iteration.

```python
        except Exception as e:
            print(f"[ERROR][HAL-911] Error inside control step loop: {e}")
            traceback.print_exc()
            raise
```

Same defensive pattern as elsewhere — log with full traceback, then re-raise so the program still stops rather than silently limping on with a broken tick.

`run_simulation()` (lines 821–894) — the outer loop that actually calls everything

```python
def run_simulation(open_space: bool = False):
    runner = CarlSimulationRunner(open_space=open_space)

    physics_ticks_per_control = 8
    control_period = 1.0 / 30.0
```

Builds the runner (which itself builds the whole HAL, reservoir, endocrine system, etc., as covered above). `control_period = 1/30` matches `self.dt`. `physics_ticks_per_control = 8` means the underlying MuJoCo physics engine steps **8 times** for every *one* cognitive control decision — physics needs a much finer timestep than 30Hz to remain numerically stable (rigid body contact dynamics are notoriously sensitive to timestep size), but the brain doesn't need to "think" that fast, so the control loop only re-evaluates sensing/ cognition once every 8 physics steps.

```python
    print("\n" + "=" * 64)
    ... (banner) ...
```

Purely cosmetic startup banner summarizing the configuration — no functional effect.

```python
```

```
    start_time = time.time()
    next_control_time = start_time
    step_durations = []

    def sync_viewer_rays():
        if runner.hal.viewer and runner.hal.viewer.is_running():
            runner.hal.viewer.user_scn.ngeom = 0
            for p1, p2, hit in runner.hal.lidar_rays:
                if runner.hal.viewer.user_scn.ngeom >= runner.hal.viewer.user
                    break
                color = np.array([1, 0, 0, 1]) if hit else np.array([0, 1, 0,
                mujoco.mjv_initGeom(...)
                mujoco.mjv_connector(...)
                runner.hal.viewer.user_scn.ngeom += 1
            runner.hal.viewer.sync()
```

A nested helper function (closure over `runner`) that draws the sonar rays as visible lines in the 3D viewer for debugging/visualization — red (`[1,0,0,1]`) if a ray hit something, translucent green (`[0,1,0.5,0.5]`) if it reached max range without hitting anything. Purely visual, has zero effect on the actual simulation logic.

python

```
    try:
        while runner.hal.viewer.is_running():
            loop_start = time.time()

            if loop_start >= next_control_time:
                runner.step()

                for _ in range(physics_ticks_per_control):
                    mujoco.mj_step(runner.hal.model, runner.hal.data)

                sync_viewer_rays()
```

The actual main loop. Runs as long as the visualization window is open. This is a manual, fixed-rate scheduler: only call `runner.step()` (one cognitive tick) once real elapsed time has caught up to `next_control_time`. When it does: run one full cognitive step, then advance physics 8 times (matching the sub-stepping explained above), then update the visual sonar-ray overlay.

python

```
            duration_ms = (time.time() - loop_start) * 1000.0
            step_durations.append(duration_ms)
            if len(step_durations) >= 90:
                avg_ms = sum(step_durations) / len(step_durations)
                max_ms = max(step_durations)
                print(f"[PROFILE] Control Loop: Avg {avg_ms:.1f} ms/tick,
                      f"Max {max_ms:.1f} ms/tick (Budget: 33.3 ms)")
                step_durations = []

            next_control_time += control_period
        else:
            time.sleep(0.001)
```

Performance profiling: tracks how long each tick actually took, and every 90 ticks (3 seconds), prints the average and worst-case tick duration against the "budget" of 33.3ms (the reciprocal of the 30Hz target rate — if a tick regularly takes longer than this, the control loop can't keep up with real-time). `next_control_time += control_period` schedules the *next* control tick a fixed interval after the *previous scheduled* time (not after `time.time()` right now) — this is important: it prevents drift, since scheduling relative to "now" would cause the effective rate to slowly drop if any tick runs even slightly over budget, whereas incrementing a fixed schedule keeps the target cadence consistent even if individual ticks occasionally run long (they'll just eat into the sleep margin rather than pushing the whole schedule later). If it's not yet time for the next control tick, sleep for 1ms and check again — a simple busy-wait-with-yield pattern rather than computing an exact sleep duration.

python

```
    except KeyboardInterrupt:
        print("\n[HAL-911] Shutdown requested.")
    finally:
        runner.reservoir.save(os.path.join(BASE_DIR, "memory", "carl_reservoi
        runner.telemetry.close()
        print(f"[GIS-901] Session complete. Food eaten: {runner.food_eaten_to
              f"RLS updates: {runner.reservoir.total_rls_updates}.")
```

Catches Ctrl+C specifically for a clean shutdown message, but the `finally` block runs *regardless* of how the loop ends (clean Ctrl+C, the viewer window being closed, or even an uncaught exception propagating up from `runner.step()`) — always saves the reservoir's current learned weights to disk and closes the telemetry file properly, so you never lose a training session's progress just because you closed the window instead of hitting Ctrl+C.

Lines 897–902: script entry point

```python
if __name__ == '__main__':
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument('--open-space', action='store_true', help='Train in c
    args = parser.parse_args()
    run_simulation(open_space=args.open_space)
```

Standard Python idiom: this block only runs when the file is executed directly (`python carl_simulation.py`), not when imported as a module elsewhere (which is exactly how `phase18_alife_pretrained.py`'s CI smoke test and other tooling can safely `import carl_simulation` without accidentally launching a full simulation). Sets up one optional command-line flag, `--open-space`, which threads through to the wall-removal logic seen way back in `SimulationHAL.__init__`.

Summary — what this file actually proves, end to end

Reading it fully top to bottom: this is a **real, working 30Hz sensorimotor control loop** with genuine physics (MuJoCo), genuine vision inference (a real trained/trainable neural network, not a stub), a genuine trainable recurrent reservoir with online learning (RLS, properly rate-limited), a rule-based hormonal state machine governing three distinct behavioral modes, and a hippocampal spatial-memory system doing real graph construction and reward diffusion. The two honest gaps visible directly in the code (not just claimed in docs) are: **the arm actuators are explicitly zeroed every tick** (lines 217, 220–221), and **the reflex layer's reservoir-bias input channel is always zero** (line 655) — both consistent with what `AGENTS.md` already told you, but now you've seen the exact lines where each disconnection actually lives.